

15 Сервис хандалгат программчлалын архитектур

15.1 Үйлчилгээний хэрэгцээ, шаардлага

15.1.1 Хуваарилагдсан программ

14-р бүлэгт бид хуваарилагдсан программчлал нь зэрэгцээ программууд бодитоор тархан байрласан өөр өөр бүтэц дээр ажиллах зэрэгцээ программчлалын төрөл болохыг харсан.

Бодит тархсан байдал бол хуваарилагдсан программ бүтээх туршлага нь "үйлчилгээ"-нд чиглэсэн тусгай программчлалын парадигмын тодорхойлолтыг хэрхэн бий болгож буй ойлгох түлхүүр юм.

Хуваарилагдсан программчлал нь дэлхийн өнцөг булан бүрт байгаа хүмүүстэй медиа (мессеж, гэрэл зураг, видео) мэдээлэл солилцох, мөн түүнчлэн бүтээгдэхүүн, үйлчилгээнд онлайнаар хандах, захиалах, төлбөр хийх боломжийг хэвийн зүйл (бид үүнийг хэрэглээ гэж үздэг шиг) болгосон. Гэсэн хэдий ч хуваарилагдсан системийг программчлах практик нь тодорхой бэрхшээлтэй тулгардаг. Үнэн хэрэгтээ, дундын санах ойн загвараас татгалзаж, ямар ч хоёр процесс бие биетэйгээ "ярилцдаг" шийдэлтэй байгаа нь (14-р бүлэгт товч танилцуулсан) тархсан программуудын зөв эсэхийг шалгахад хэцүү болгодог (жишээ нь, бүрдмэл нь төрлийн аюулгүй).

Практик жишээг авч үзвэл, *локал* програмууд (бид хуваарилагдсан бүс программыг илэрхийлэхийн тулд бүлгийн турш энэ нэр томъёог ашиглана) - програмын модулиудыг угсарч, модулиуд нь хэрэгжилтэнд амласан интерфейстэйгээ нийцэж байгаа эсэхийг шалгах замаар Өгөгдлийн хийсвэрлэлийг (9-р бүлгийг үзнэ үү) эргэн харья. Энэ нь бид программ бүхэлдээ нийцэж байгаа эсэхийг шалгах боломжтой (статик эсвэл динамик, 8.8-р хэсгийг үзнэ үү). Энэ таамаглалыг ерөнхийдөө хуваарилагдсан программд авч үздэг. 14.5-д дурдсанчлан, процессууд мессеж солилцоход чиглэсэн төрөл бүрийн техникийг хялбархан параметржуулж болох бөгөөд ингэснээр дурын хоёр процесс динамикаар харилцан үйлчилж, мессеж солилцох боломжтой болно. Энэхүү динамик нэгтгэлд процесс нь бие биенийхээ мессежийг гэрчилдэг цорын ганц зүйл бөгөөд энэ нь дагаж мөрдөх олон талт асуудлыг бий болгодог.

- Мессеж нь найдвартай эсвэл найдваргүй байдлаар дамжуулагдаж байна уу?
- Програмууд мессежийн эрэмбийг адилхан харж байна уу (жишээ нь, илгээгчийн хардаг шиг)?
- Хүлээн авагч мессежийг дахин эрэмбэлж болох уу?
- Мессежүүд нь хүлээн авагчийн байх ёстой форматтай байна уу (тэдгээрийг `parse` хийх боломжтой юу)?
- Мессежүүд нь хүлээн авагчийн харгалзуулсан үйлдлийг дуудаж байна уу?
- Холбогдох үйлдлийн төрлүүдтэй мессежид агуулагдсан өгөгдөл нийцэж байна уу?
- Илгээгч болон хүлээн авагч алдаатай хэрхэн харьцах ёстой вэ (тэд үүнийг нөгөө талдаа мэдэгдэх шаардлагатай юу)?

Өмнөх бүлгүүдэд бид локал програмуудад нийцлийн баталгааг (статик эсвэл динамик) өгөх арга техникийг үзсэн бөгөөд тэдгээр нь ижил цагт ажиллах тул дуудагч болон дуудуулагчийн хоорондох дуудлагын эрэмбийг хадгалах гэх мэт өөрчлөгдөхгүй гэж үзэж болно. Тархсан програмуудад дуудагч болон дуудуулагч нь өөр өөр машин дээр ажиллахаас гадна харилцааны протоколууд нь мессеж хэрхэн солилцохыг тодорхойлдог. Түүнчлэн, тэдгээр нь нэгэн ижил ойлголт

болон өгөгдлийн бүтцийг илэрхийлэх өөр өөр арга замаар (жишээ нь, өөр өөр програмчлалын хэл ашиглан хэрэгжүүлэх үед) өөр өөр цагт ажиллуулж болно. Энэ бол нийцлийн баталгааг хангахын тулд хэл шинжлэлийн арга техникээр эдгээр элементүүдийн заримыг нь тохируулах ёстой (жишээ нь: харилцаа нь эрэмбэ хадгалдаг, алдаа нь онцгой тохиолдол мэт ажилладаг, 7.3-р хэсгийг үзнэ үү) эсвэл эдгээрийг бүрдэлийн үүднээс шаардагддаг байх ёстой гэсэн үг юм.

Бодит жишээ болгон бид client-ээс ирсэн мессежийг порт дээр хүлээдэг server бичиж болно. Энэ солилцоо нь бид ямар ч статик нийцлийг шалгах боломжгүй байх динамик модуль нэгтгэлийг төлөөлдөг (Бид server-тэй холбогдох хүртэл client-ийн хэн болохыг мэдэхгүй). Мэдээжийн хэрэг, хэрэв server нь client-ээс төлөвлөөгүй мессеж хүлээн авбал (жишээ нь, түүний хийдэггүй үйлдлийн талаар) client нь хэрэгжүүлэхээр тохиролцсон интерфэйсийг дагахгүй байна гэж server дүгнэж болно. Гэсэн хэдий ч, энэ шалгалт нь зөвхөн server-т хийгдэх бөгөөд ийм тохиолдлын талаар бусад модульд мэдэгдэх нь утгагүй, жишээлбэл, хэрэв client ийм тохиолдолд хэрэгжүүлэлт өмнөх тохиролцоонд (интерфэйс) байхгүй л бол client-д мэдэгдэх боломжгүй байх болно.

Тусгай зориулалтын хийсвэрлэл, техник, технологитой "үйлчилгээ хандалгат программчлал" гэсэн нэр томъёонд хамаарах зарчмаар хөгжүүлэхийг шаарддаг эдгээр сорилтууд нь програмистуудад тархсан програмуудын тогтолцооны шинж чанарыг тодорхойлох, шаардлага тавьж, улмаар шалгахад тусалдаг.

15.1.2 Нээлттэй системүүд

Түүхийн үүднээс авч үзвэл, хүмүүс болон бизнес хоорондын дижитал мэдээлэл солилцоонд суурилсан дижитал эдийн засаг бий болсноор, мөн сүлжээний холболт болон ард нь хоорондоо холбогдсон компьютеруудын байх, жишээлбэл, World Wide Web болон цахим шуудан зэрэг глобал системийн интернетийн өргөжилтийн ачаар 1990-ээд онд хуваарилагдсан програмчлал маш их алдартай болсон.

Дэлхий даяар хэрэглэхийн тулд хуваарилагдсан программын хөгжүүлэгчид тархсан програмчлалын нээлттэй хандлагыг дагаж мөрдөх шаардлагатай байв. Тэдний хэрэглэгчид харилцан үйлчлэх, холбогдох боломжийг олгохын тулд харилцан ажиллах программ хэрэгтэй байв. Тухайн программын бүрдэл хэсгүүд, тэдгээрийн ажиллах хугацаа тогтмол байдаг хаалттай хандлагыг баримтлан тайлбарлах боломжтой; Энэ нь локал програмуудтай адил нийцлийн шалгалтыг дэмжих давуу талтай бөгөөд энэ нь дэлхий даяар тархсан сая сая (дараа нь хэдэн тэрбум) хэрэглэгчдийн байнга өсөн нэмэгдэж буй хэрэгцээг хангахад хэцүү байх болно. Жишээлбэл, энэ арга нь бид вэб сайт гэх мэт нэгж бүрдэл хэсгээр өгдөг, шинэчилдэг учраас л интернетээр харилцдаг бүх программыг (дэлхий даяар тархсан бүх компьютерт) шинэчлэх үндэслэлгүй хэрэгцээг илэрхийлнэ.

Тиймээс интернетийн өргөжилтөд саад учруулахгүйн тулд хөгжүүлэгчид тархсан программыг хэрэгжүүлэх нээлттэй арга барицыг хэрэгжүүлэх шаардлагатай болсон. Тодруулбал, тэд өөрсдийн программ хангамжийн бүрдэл хэсгүүдийг өөрсдийн бичээгүй, хэрэгжүүлэлтийг нарийн мэдэхгүйгээр бусад бүрдэл хэсгүүдтэй харилцах боломжийг олгодог стандартуудыг тодорхойлж, батлах шаардлагатай байв.

Бидний өдөр тутмын амьдралаас жишээ болгон дурдахад бид төхөөрөмждөө суулгасан вэб хөтчөө мэдээллийн хуудас, худалдааны жагсаалт эсвэл хайлтын илэрцэд нэвтрэх боломжийг олгодог вэб серверт холбох бүртээ нээлттэй, тархсан системийг угсардаг. Манай вэб хөтчүүд дээр суурилсан олон стандартуудын дотроос бид Internet Protocol (IP), the Transmission Control Protocol

(TCP), the Hypertext Transfer Protocol (HTTP), the Extensible Markup Language (XML), and the JavaScript Object Notation (JSON) олдог.

15.1.3 Сүл холболт ба харилцан ажиллах чадвар

Дүгнэж хэлэхэд, тархсан программуудын холбогдох порт хэсэг нь нээлттэй байх шаардлагатай. Ийм нээлттэй байдлыг дэмждэг хоёр үндсэн зарчим бол *сүл холболт* ба *харилцан ажиллах чадвар* юм.

Сүл холболт нь программыг байгуулж буй модулиудын хоорондын холбоо сүл байгааг харуулж байгаа тул модулийн дотоод өөрчлөлтийг программын бусад модулиуд ажиглах боломжгүй. Тархсан орчинд, объектуудад харагдах хийсвэрлэлийн зарчмыг өргөтгөдөг (10.2.3-ыг үзнэ үү) - энэ нь модулиуд бусад модулиудын дотоод дэлгэрэнгүйн тухай маш бага мэдлэгтэй байдаг, тиймээс тэдгээрийг хэрэгжүүлэхэд ашигладаг технологийн талаархи өндөр эрх чөлөө гэсэн үг юм (жишээ нь: техник хангамж, гүйцэтгэх орчин, хэрэгжүүлэх хэл, логик).

Нөгөө хэсэг нь харилцан ажиллах чадвартай холбоотой. Бидэнд хоорондын сүл холбоотой модулиуд байгаа бөгөөд тэдгээрийг хэрэгжүүлэхэд өөр өөр технологи ашиглах боломжтой. Одоо тархсан программынхаа логикийг ойлгохын тулд бид эдгээр модулиудад программыг бүрдүүлдэг бусад модулиудын үйл ажиллагааг идэвхжүүлж, мэдээлэл солилцох боломжийг олгох хэрэгтэй. Холболт ба харилцан ажиллах чадвар нь хоорондоо холбоотой ойлголт юм. Хүчтэй холбогдсон модулиуд нь угаасаа харилцан ажиллах чадвартай байдаг (жишээлбэл, объект өөр объектын методыг дууддаг, үүн рүү санах ойн зарим өгөгдлийн хаягийг илгээдэг гэж төсөөлөөд үз дээ). Учир нь хоорондоо нягт уялдаатай модулиуд нь модулиуд хэрхэн харилцан үйлчлэлцэх, өгөгдлийг хэрхэн илэрхийлэх, зөөх, хадгалах гэх мэт хэрэгжилтийн технологиос заасан олон тооны харилцан ажиллах чадварыг далд хэлбэрээр дундаа ашигладаг. Эсрэгээр, сүл холбогдсон модулиуд нь харилцан ажиллах боломжтой сонголтыг ил байлгах хэрэгтэй, жишээлбэл, нэг модуль нь нөгөө модулийн үйлдлийг хэрхэн идэвхжүүлэх (жишээ нь, дамжуулалтын ямар технологи гэх мэт) болон өгөгдөл дамжуулах (жишээ нь, өгөгдлийн ямар формат гэх мэт).

Харилцан ажиллах чадвартай холбоотой шийдвэрүүд нь нэг тархсан программд төвлөрч байхад тархсан нээлттэй системтэй болохын тулд хөгжүүлэгчид өөрсдийн модуль хоорондын харилцан үйлчлэх дотоод дүрэм гаргадаг боловч харилцан ажиллах чадварын стандар технологиуд бидэнд хэрэгтэй. Ийм байдлаар хөгжүүлэгчид харилцан ажиллах боломжтой технологийн стек бүхий ямар ч модулийг тархсан программдаа нэгтгэх боломжтой. Үүний нэгэн адил, өөр өөр тархсан программууд харилцан ажиллах боломжтой л бол харилцан үйлчлүүлж болно.

Үйлчилгээн хандалгат програмчлалд зоиулсан хэл шинжлэлийн хийсвэрлэл, технологийг авч үзэхэд бид эдгээр нь үйлчилгээний сүл холболт, харилцан ажиллах чадварыг олж авах, дэмжихэд хэрхэн оролцдогийг харах болно.

15.1.4 Үйлчилгээ хандлага

Хэдийгээр үйлчилгээ хандалгат парадигмын талаархи анхны санаанууд нь 2000-аад оны эхэн үеэс гарч ирсэн боловч компьютерын шинжлэх ухаанд *үйлчилгээний (service)* тухай ойлголт нь үйлчилгээний чиг хандлагаас өмнө гарч ирсэн. Үнэн хэрэгтээ бид парадигмын өмнөх контекстээс үйлчилгээ хандалгат програмчлалын зарим шинж чанаруудтай холбоотой "үйлчилгээ" гэсэн нэрийг харж болно.

Жишээлбэл, давхаргуудын харилцан үйлчлэлээр шинж чанартай сүлжээний технологийн олон жишээ байх бөгөөд тэдгээр нь тус бүртээ харилцааны тодорхой асуудлыг шийдвэрлэхэд зориулагдсан байдаг. Сүлжээний стек хөгжүүлэгчид эдгээр давхаргыг хийсвэр машин гэж үздэг бөгөөд тус бүр нь эдгээр функцуудыг зэргэлдээх давхаргад үйлчилгээ болгон өгдөг. Жишээ нь, давхарга нь сигнал гаргадаг, хүлээн авдаг физик техник хангамжийг удирддаг бөгөөд хоёр үйлдлийг гүйцэтгэх боломжтой; Нэг нь тодорхой хэмжээний сигнал хүлээн авмагц массив битийг буцаадаг (тэдгээрийг дохионы дараалал болгон гаргах тохиролцоотой) ба нөгөө нь массив битийг хүлээн авдаг. Саяын тайлбарласан давхаргын зэргэлдээх давхарга нь packet-switching логикийг хэрэгжүүлэхийн тулд энэхүү "биет" үйлчилгээг ашиглана.

Үүний нэгэн адил үйлдлийн системээс хэрэглэгчдэд үйлчилгээ үзүүлэхийн тулд үйлдлийн системээс ажиллуулдаг процессууд болох (nomen omen) Windows үйлчилгээнүүд болон Unix daemons-ыг (daemon үрсгалтай андуурч болохгүй, 14.7.2-ыг үзнэ үү) олдог. Үлгэр жишээ Unix daemon бол sshd бөгөөд хэрэглэгчдэд алсын зайнаас хандах болон командыг гүйцэтгэх боломжийг олгодог программ юм. Өөр нэг нь D-Bus процесс хоорондын харилцааны үйлчилгээний лавлагааны хэрэгжилтийн бүрдэл хэсэг болох dbus-daemon бөгөөд процессууд нь харилцааны шугамаар холбогдсон мэт хоорондоо харилцах боломжийг олгодог – daemon нь шугамд холбогдсон аливаа процессоос мессежийг чиглүүлдэг.

9.2 ба 10.2.3-ын агуулгыг санаж, үйлчилгээнд чиглэх энэхүү хандлага нь биднийг огт гайхшруулахгүй; Энэ нь бид программ хангамжийн системийн хүндрэлийг модуль болгон задалж, бусад модулиудад үйлдлийн багцыг ил гаргаж, дотоод логик болон төлөвийн дүрслэл гэх мэт дэлгэрэнгүйг далдалтаг мэдээллийг нуух, өгөгдлийн капсулжуулалтын тухай ойлголтуудын хэрэглээ юм.

15.2 Сервис Хандалгат программчлалын элементүүд

15.2.1 Үйлчилгээ, Давхаргжуулсан харагдах байдал

Програм хангамжийн нэгжийг үйлчилгээ гэж үздэг хуваарилагдсан программыг бий болгох конвенц, практик байдаг хэдий ч үйлчилгээ хандалга бий болсноор үйлчилгээ нь тусгай программчлалын парадигмын нэгдүгээр зэрэглэлд гарч ирсэн юм.

Үйлчилгээ хандалгаар бол *үйлчилгээ (сервис)* нь мессеж солилцоогоор үйлдлийг санал болгодог бие даасан процесс юм. Энэ бол үйлчилгээ нь өөрийн дотоод төлөвийг (мөн дүрслэлийг) далдалдаг гэсэн үг бөгөөд хэрэглэгчдэд зориулсан цорын ганц хандалтын гадаргуу нь үйлдлээр хангагдсан, мессеж дээр суурилсан дуудалтаар хандах боломжтой гэсэн үг юм.

Сервис ба Объект

Үйлчилгээн дээрх тодорхойлолт нь зарим уншигчдад объект хандалгатаар объектын тодорхойлолтыг төсөөлөгдөж байгаа бол тэд бүрүү замаар яваагүй. Үнэхээр, 10.2.1-д заасанчлан объект нь зарим өгөгдөл болон түүнийг удирдаж буй үйлдлүүдийг багтаасан капсул бөгөөд тухайн объектын хэрэглэгчид зөвхөн нийтийн интерфэйс дээр цуглуулсан эдгээр үйлдлүүдийг мессежээр дамжуулан дуудах замаар эхнийхтэй харилцах боломжтой.

Үзэл баримтлалын хувьд объект болон үйлчилгээний хоорондох ялгаа бол объект нь бие даасан процесс болдоггүйтэй холбоотой байдаг тул харьцангуй нимгэн байдаг. Гэсэн хэдий ч энэхүү бяцхан үзэл баримтлалын зөрүү нь практикт ихээхэн өөрчлөлт авчирдаг.

Жишээ нь, 10.3.1-д харснаар гүйцэтгэлийн ашгийн шаардлагаар объект хандалтат хэл дээрх "мессежээр дамжуулан дуудах" гэсэн үндсэн тайлбар нь тухайн метод/үйлэлтэй холбоотой кодыг ажиллуулж, түүнд боломжит параметруудийг санах ойн хаяг болгон дамжуулдаг үр ашигтай процедурыг хэрэгжүүлдэг.

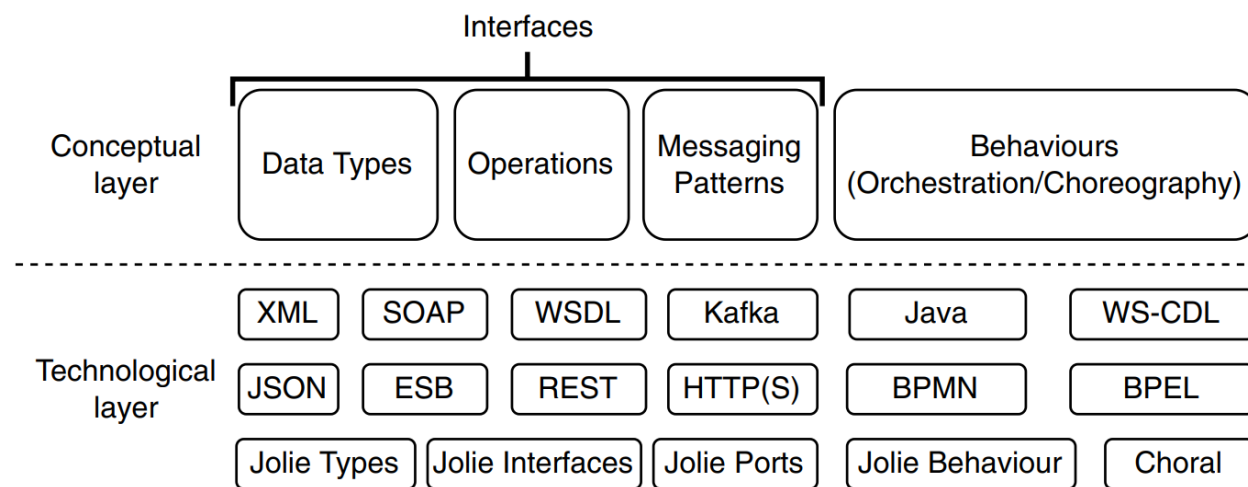
Хэрэв нэг талаас энэ тайлбар нь хэрэгжсэн объект хандалтат хэлийг үр дүнтэй болгож байгаа бол нөгөө талаас програмистууд объектуудыг санах ойноос өөр хэрэгслээр бие биетэйгээ харьцах боломжийг олгох боломжгүй болгодог - Алсын процедурын дуудлагын бүдүүвч зэрэг ad-hoc процедур хэрэгжүүлэх (14.5.4-р хэсгийг харна уу) эсвэл Java алсын метод дуудалт гэх мэт технологийн тусгай шийдлүүдийг ашиглах (14.7.3-ыг үзнэ үү).

Үйлчилгээний бие даасан байдлын зарчим нь технологийн онцлогт тохирсон оновчлолын аль нэгийг эвддэг бөгөөд энэ нь хөгжүүлэгчдэд (болон хэлний дизайнерууд) эхлээд үйлчилгээг тодорхойлох үзэл баримтлалын шинж чанаруудыг, дараа нь нь үүнийг хэрэгжүүлэхэд ашиглаж болох технологийн стэкийг авч үзэхийг шаарддаг.

Үйлчилгээний үзэл баримтлал ба технологийн давхаргын хоорондын хамаарлыг 15.1-р зурагт үзүүлэв. Зураг дээр нэгжийн хоёр хэмжээст байрлал нь давхарга хооронд бие биетэйгээ холбоотой элементүүдийг хадгалдаг зохицуулалтыг дагадаг (мэдээж технологийн хувьд бүрэн гүйцэд биш).

Ойлголтын давхарга

Үзэл баримтлалын давхарга нь үйлчилгээг тодорхойлох онцлогуудыг хийсвэрээр тайлбарлаж, дараа нь тэдгээр шинж чанаруудыг хэрэгжүүлэхэд ашиглаж болох өөр өөр технологийн багцыг тодорхойлж, өргөтгөн байдрлаар үйлчилгээгээ тодорхойлдог.



Зураг 15.1 Үйлчилгээ хандалгат программчлалын давхаргууд

15.1-р зурагт үзүүлснээр бид концепцийн давхаргын элементүүдийг хоёр багц болгон хувааж болно. Нэг нь *интерфэйсүүд* бөгөөд бид үйлчилгээний хэрэглэгчдэд гаргасан нэгжүүд; түүний гүйцэтгэхийг хүсэж буй үйлдлүүд, хэрэглэгчид тэдгээрийг хэрхэн дуудах (жишээ нь, тэдний мессежийн загвар) буюу хүлээн авч, өгөхийг заасан өгөгдлийн хэлбэр. Нөгөө багц нь хэрэглэгчийн хүсэлтийн дагуу хариуг тооцоолох, олон талын зорилгыг (multi-party) биелүүлэхийн тулд бусад

үйлчилгээг дуудах гэх мэт дотооддоо ашигладаг үйлчилгээний логикийг харгалзан үздэг *орчил (behaviors)* юм.

Интерфэйс

Хийсвэр өгөгдлийн төрөл болон объект дээрх шиг интерфэйсүүд нь хэрэглэгчдэд үзүүлэх нийтийн харагдацыг илэрхийлдэг. Интерфейсийн "өгөгдлийн төрлүүд" нь үйлдлийг идэвхжүүлэх үед хүлээн авахаар заасан ба/эсвэл гаргахаар амласан (promises) өгөгдлийн бүтцийг дүрсэлдэг. Үйлчилгээний санал болгож буй "үйлдэл" нь дуудагдах боломжтой шошгоны багц байна. Үйлчилгээнүүд нь объектын метод гэх мэт (энэ нь эргээд 7-р бүлэгт авч үзсэн нийтлэг стект суурилсан хяналт-шилжүүлэх загварт нийцдэг) хүсэлт-хариултын "мессежийн загвар"-ыг дагаж мөрддөггүй гэдгийг онцлон тэмдэглэх нь зүйтэй. Эсрэгээр, дуудлагад ашигласан нэг буюу хэд хэдэн өгөгдлийн бүтэцтэй холбоотой байхаас гадна үйлдлүүд нь хэрэглэгчидтэйгээ мессеж солилцох арга замыг тусгайлан зааж өгдөг. Жишээлбэл, үйлдлийн дуудлалт нь нэг талын (one-way) мессеж солилцох найдваргүй хэв маягийг дагаж мөрддөг бөгөөд дуудагч нь үйлчилгээнээс хүлээн авах, ашиглах баталгаагүйгээр мессежээ үйлчилгээ рүү илгээдэг. Өөр нэг хувилбар бол нэг талын найдвартай солилцоо бөгөөд дуудагч нь үйлчилгээнээс мессеж хүлээн авалтын (гэхдээ түүний хэрэглээний тухай биш) мэдэгдэл авахыг хүлээж байдаг.

Орчил

Үзэл баримтлалын давхаргын сүүлийн элемент болох орчил (behaviors) нь үйлчилгээний төлөв ба интерфэйсийг холбодог.

Энэ үзэл баримтлалыг нэвтрүүлэхийн тулд объектын зэрэгцээ бус загваруудыг сүүлчийн удаа харцгаая. Объектууд төлөвийг хамруулж байгаа хэдий ч сүүлийнх нь тухайн объект өөрөө аль ч аргын дуудлагад хариу өгөх боломжид нөлөөлөхгүй—сайндаа нь бид зарим алдаа эсвэл онцгой тохиолдлыг өдөөж болох ч дуудалт үргэлж боломжтой байдаг. Үйлчилгээнд хөгжүүлэгч нь мессеж хүлээж авахаар үйлчилгээг бичиж болно, жнь: count үйлдлийн дуудалтыг идэвхжүүлдэг access үйлдэлт - тухайн сервисээр бүртгэгдсэн accesses-ийн буй тоог хэрэглэгчид буцаадаг. 14.5.3-т дурдсанчлан хэрэв хэрэглэгч хоёр үйлдлийн шугаманчлал (linearization)-ыг мэдэхгүй бол тэдний харилцан үйлчлэл түгжрэлд орж болзошгүй. Жишээлбэл, үйлчилгээ нь эхлээд access-ыг хэрэглэж, дараа нь (хэрэглэж) count-д хариу өгнө гэж үзвэл хэрэглэгч хоёр дуудлагыг зэрэгцүүлэн илгээж болно. Харин ч эсрэгээрээ, хэрэв а) детерминизмын улмаас үйлчилгээ нь эхлээд count дуудалтыг хүлээн авч, дараа нь access хийх боломжтой б) үйлчилгээ нь мессежийг FIFO (14.4.2-ыг үзнэ үү) дарааллаар эрэмбэлсэн гэж үздэг - ингэснээр мессежийг зөвхөн ирэх дарааллаар нь ашигладаг бөгөөд энэ хоёрын хоорондын харилцан үйлчлэл түгжирдэг.

Ажиллагааны хувьд үйлчилгээ хандалгат программчлал нь үйлчилгээний хуваарилагдсан логикийг хэрэгжүүлэх хоёр аргатай: *orchestration* ба *choreography*. Өмнөх хэсэг дэх олон "үйлчилгээ" чухал байдаг бөгөөд учир нь нэг үйлчилгээ нь зарим нэг ажлыг гүйцэтгэхэд хангалтгүй, харин хоёр ба түүнээс дээш үйлчилгээний харилцан үйлчлэлийг шаарддаг үед үйлчилгээний логикийг хэрхэн програмчлах тухай асуудал хамааралтай болно.

Жишээлбэл, цахим худалдааны гүйлгээг хэрэгжүүлэхийн тулд user үйлчлүүлэгчтэй харилцах шаардлагатай shop, bank үйлчилгээг төсөөлөөд үз дээ. Тэнд гүйлгээний логикийг хоёр үйлчилгээний аль нэгээр нь хянах эсвэл тэдгээрийн хооронд хуваарилах боломжтой.

Оркестрийн хувьд *orchestrator* гэж нэрлэгддэг нэг үйлчилгээ нь бүрдэлд оролцож буй бусад үйлчилгээг зохицуулах замаар тархсан ажиллагааг хэрэгжүүлдэг. Жишээлбэл, хэрэв бид shop-ийг найруулагч (*orchestrator*)-аар сонговол, тухайлбал, хэрэглэгчээс хаягийг нь асуух, bank-ны шилжүүлгийн захиалгыг бэлтгэх гэх мэт user болон bank-тай харилцах харилцааг зохицуулах болно. Энд user болон bank хоёр идэвхгүй (*passive*) үүрэг гүйцэтгэдэг, өөрөөр хэлбэл тэд үргэлж бэлэн байх (хамгийн тохиромжтой) үндсэн үйлдлүүдийг өгдөг бөгөөд найруулагч (*shop*) гүйлгээний хуваарилагдсан логикийг хэрэгжүүлэхийн тулд дууддаг. Дүгнэж хэлэхэд, *orchestration*-д хуваарилагдсан логикийг голчлон *orchestrator* тодорхойлж, гүйцэтгэдэг бөгөөд энэ нь бүрдэлд оролцдог бусад үйлчилгээнүүдийн хоорондын бүх харилцан үйлчлэлийг зохицуулж, зуучилдаг.

Оркестржуулалт нь нэг үйлчилгээнд хуваарилагдсан бүрдлийн логикийг төвлөрүүлэх сайн шинж чанартай байдаг ба программистууд үүнийг аль алинд нь хэрэгжүүлэх, зарим нэг лавлагааны тодорхойлолтын дагуу зөв эсэхийг шалгахад хялбар болгодог - оркестр нь нэг хяналтын цэг юм. Энэ нь функционал болон гүйцэтгэлийн ашгийн аль алинд нь асуудал үүсгэж болно. Жишээлбэл, найруулагч нь зохицуулдаг үйлчилгээнүүдийн хооронд дамжуулж буй бүх өгөгдлийг харж, бусад үйлчилгээнүүд ямар үйлдлүүдийг гүйцэтгэх ёстойг нэг талдаа зааж өгч болно. Хэрэв манай программд хэсгүүд нь төв найруулагчид "right thing" хийх гэж итгэхийг зөвшөөрөхгүй байх функциональ шаардлага байгаа бол бид Оркестржуулалттай болох боломжгүй - "Хэрэглэгч-банк-дэлгүүр"-ийн жишээн дээр нэг бүтээгдэхүүн худалдан авахдаа хэрэглэгчээс хоёр дахин их хэмжээний мөнгө төлдөг хуурамч дэлгүүрийг төсөөлөөд үз дээ. Гүйцэтгэлийн ашиг талаас харахад оркестржуулалт нь үйлчилгээ хооронд солилцсон мессежийн тоог нэмэгдүүлнэ гэсэн үг юм. Жишээлбэл, *shop* найруулагч нь *bank* ба *user* хоорондын харилцан үйлчлэлийг зуучилж байгаа тул эхнийхээс нөгөө рүү шууд мессеж илгээхийн оронд бидэнд гурван алхам байна: (1) *shop* нь *bank*-аас *user*-д мессеж өгөхийг хүсэх, (2) *bank* мессежээр хариу өгөх, (3) *shop* нь *shop* рүү мессеж илгээх. Дамжуулах зурвасын өргөн хязгаарлагдмал, хоцрогдол ихтэй эсвэл мессежийн ачаалал ихтэй үед солилцсон мессежийн тоо нэмэгдэх нь тархсан программын гүйцэтгэлийн ашиг, хэрэглээнд ихээхэн нөлөөлдөг.

Тиймээс бид бүхэл бүтэн тархсан харилцан үйлчлэлийг зуучлах найруулагчийг тодорхой олж чадахгүй хувилбарууд байдаг. Энэ тохиолдолд үйлчилгээ нь гүйлгээний тархсан логикийг дэглэлт (*choreography*) болгон хэрэгжүүлэх ёстой, өөрөөр хэлбэл оролцогч бүр харилцан үйлчлэлд өөрийн хэсгийг мэддэг, хэрэгжүүлдэг байдлаар ажиллуулах ёстой. "Дэглэлт" гэсэн нэр нь бүжиг дэглээчтэй жүжигчилсэн тоглолтын нэгэн адил бүжгийн үзүүлбэр бүр тодорхой үүрэг гүйцэтгэж, тухайн дүрийн гүйцэтгэх ёстой үйлдлүүдийг гүйцэтгэдэг учраас бий болсон. Бид оркестржуулалт ба дэглэлтийн хоёрдмол байдлыг нэг үйлчилгээ нь бусад үйлчилгээг зохицуулах замаар бүхэлд нь хуваарилагдсан программын логикийг бий болгох, үйлчилгээнүүд бие даасан харилцан үйлчлэлээр дамжуулан уг логикийг бий болгох хоёрын ялгаа гэж харж болно. Энэ тохиолдолд дэглээч нь өөрөө үйлчилгээ биш (оркестратороос ялгаатай), харин дэглэлтийг тодорхойлсон нэгж (жишээ нь, программист) юм.

Дэглэлтүүд нь хэдийгээр ингэж нэрлэгдээгүй ч үйлчилгээ хандалгаас өмнө үүссэн бөгөөд өөр өөр талуудын харилцан үйлчлэлийн дарааллыг тодорхойлоход хэрэглэгддэг загвартай төстэй юм (Компьютерийн шинжлэх ухааны зарим эхэн үеийн жишээ бол аюулгүй байдлын протоколын Алис ба-Бобын тэмдэглэгээ, мессежийн дарааллын графикууд, дарааллын диаграмууд юм).

Оркестржуулалтын тухай ерөнхий ойлголт бий болсон ч дэглэлтийг ашиглах стандарт тайлбар одоогоор алга байна. Жишээлбэл, бид дэглэлтийг лавлагааны үзүүлэлт гэж үзэж болох

бөгөөд ингэснээр програмистууд түүнд оролцож буй үйлчилгээг зөв хэрэгжүүлэхийн тулд дэглээцийг үзэх хэрэгтэй. Өөр нэг тайлбар бол программист өөрийн үйлчилгээний хэрэгжилт нь дэглэлтэнд заасан үүрэгтэй нийцэж байгаа эсэхийг механикаар шалгахын тулд дэглээч ашигладаг дэвшилтэт төрлийнх юм. Гурав дахь тайлбар нь дэглэлтийг программ гэж үзэх явдал юм. Энэ тохиолдолд хөгжүүлэгчид оролцож буй бүх үйлчилгээний үйлдэл, харилцан үйлчлэлийг дэглэлтийн программ болгон зааж өгдөг. Дараа нь дэглээч эх код өгөгдсөн бол хөгжүүлэгчид компилятор ашигладаг бөгөөд үүнд оролцож буй бүх үйлчилгээний зөв кодыг автоматаар үүсгэдэг. Эдгээр боломжит тайлбаруудыг бид 15.2.5-д дүрсэлсэн болно. Мэдээжийн хэрэг, эдгээр тайлбарууд нь бие биенээсээ үл хамаарах зүйл биш бөгөөд жишээлбэл, дэглэлт хийсэн системийн зарим хэсгийг хэсэгчлэн хэрэгжүүлэх, үлдсэн хэсгүүдийг дараачийн шатанд хэрэглэгч өгөх боломжтой хэвлэгдсэн параметруудийг тодорхойлох гэх мэт дэглэлтийн программ, төрлүүдийг хольж болно.

Технологийн давхарга

Технологийн давхарга нь концепцийн түвшинд үйлчилгээг тодорхойлдог нэг буюу хэд хэдэн элементүүдийг шийдвэрлэх шийдлүүдийг агуулдаг. Зураг 15.1-д эдгээр технологийн бүрэн бүс жагсаалтыг харуулсан бөгөөд бид дараагийн хэсэгт авч үзэх болно.

Бидний сонирхож байгаа зүйл бол эдгээр шийдлүүдийн програмчлалын хэлтэй харилцах харилцаа бөгөөд дараах хэсгүүд нь танилцуулсан хувилбаруудын аль нэгийг нь бүхэлд нь хамрах зорилгогүй, харин хэлний бүтэц, хийсвэрлэл, дүн шинжилгээ хийх үүднээс эдгээрийн нэг буюу хэд хэдэн зүйлийг тэмдэглэсэн холбогдох ухагдахуунуудад анхаарлаа төвлөрүүлэх болно.

15.2.2 Өгөгдлийн төрөл

8-р бүлэгт өгөгдлийн төрөлд дээр үзүүлсэн асуудлууд нь мөн үйлчилгээний хандалгад байдаг. Үйлчилгээний хандлагын нэг онцлог нь гадаад хэрэглэгчдэд үзүүлж буй төрөл, утга нь үйлчилгээний дотооддоо тодорхойлсон төрөл, утгуудаас ихэвчлэн ялгаатай байдаг оршино.

Энэхүү тасалдалт нь хуваарилагдсан процессуудын хооронд өгөгдөл урсах боломжийг олгохын тулд үйлчилгээний дотор талаас өгөгдлийг санах ойн дүрслэлээс дамжуулалтанд шаардлагад тохируулсан формат руу *сериалчлах (serialization)* гэж нэрлэгдэх процедураар хувирдагаас үүсдэг бөгөөд хүлээн авсны дараа түүнийг дахин хөрвүүлэхийн тулд *сериалчлалыг буцаах (deserialization)* гэж нэрлэгддэг эсрэг процедурыг санах ойн дүрслэл болгон хувиргадаг. Мэдээжийн хэрэг, сайн өгөгдлийн формат (үүнтэй холбоотой орчуулгын процедур нь) нь тусгайлсан хэлний ажиллах хугацаа эсвэл техник хангамжийн архитектуртай холбоотой байж болохгүй бөгөөд өгөгдлийн бүтцийн семантик хамааралтай элементүүдийг үйлчилгээ хооронд шилжүүлэх баталгаа болдог.

Бодит жишээ дурдахад, хэрэв А үйлчилгээ нь бүхэл тоо гэх мэт үндсэн утгыг ч В үйлчилгээ рүү илгээхийг хүсвэл өөрийн санах ойд байгаа битүүдийг тухайн утгаар шүүд хуулж чадахгүй. Үнэн хэрэгтээ, А нь В-ийн ямар ч хэрэгжилт, архитектурын дэлгэрэнгүй мэддэггүй бөгөөд В-ийн санах ойн бүтэц нь түүний санах ойн бүтцийг байршуулж чадна (дотор нь байгаа өгөгдлийг зөвөөр илэрхийлэх) гэдэг баталгаа байхгүй.

Хүлээгдэж буйгаар эдгээр асуудал нь харилцан үйлчлэлийн, хуваарилагдсан процессуудын сул холболт, харилцан ажиллах чадварын зарчимд нийцдэг.

Тиймээс, үйлчилгээний үйлдлийг тодорхойлдог өгөгдлийн төрлийн талаар ярихдаа бид үндсэндээ хоёр хүчин зүйлийн талаар ярьдаг: дамжуулж буй өгөгдлийн бүтэц, түүнийг сериалчлахад ашигладаг өгөгдлийн формат.

Форматууд нь харилцан адилгүй боловч тэдгээрийн олонхын нийтлэг шинж чанар бол өгөгдлийн бүтэц нь мод шиг, циклгүй бүтэцтэй бичлэгүүдтэй төстэй байдаг бөгөөд энэ нь голчлон санах ойн хаягийг найдвартай илэрхийлэх, сериалчлах (буцаах) хүндрэлээс шалтгаалдаг.

Технологийн хувьд бид ерөнхийдөө хүний унших боломжтой болон хоёртын форматын хооронд хуваагдсан шийдлүүдийг хардаг. Эхнийх нь өгөгдлийг хүн уншихад хялбар, машинд хоёрдмол утгагүй задлаж болох хэвлэх тэмдэгтүүдийн дараалал болгон хувиргадаг. Сүүлийнх нь (хүний) уншигдах чадвараас татгалзаж, өгөгдлийг санах ойн ажил ба/эсвэл дамжуулалтыг оновчтой болгох битийн мөр болгон кодчилдог.

Үйлчилгээ хандалгат программчлалын хүрээнд хэрэглэгдэж буй хүний унших боломжтой хамгийн алдартай хоёр форматыг жишээ болгон ашиглаж байна: JavaScript Object Notation (JSON) болон Extensible Markup Language (XML).

Түүний товчлолоос харахад, JSON нь анх вэб хуудасны скрипт бичихэд зориулагдсан программчлалын хэл болох JavaScript хэл дээр үндэслэсэн боловч энэ формат нь үндэслэсэн хэлнээсээ хамааралгүй бөгөөд үндсэн өгөгдлийн төрлүүд болон бүтэцлэгдсэн өгөгдлийг төлөөлөх синтаксийг гаргаж авдаг.

JSON-ийн үндсэн өгөгдлийн төрлүүд нь логик (true ба false гэсэн тэмдгээр дүрслэгдсэн), тоонууд (давхар нарийвчлалтай, хөвөгч цэгийн), тэмдэгт мөрүүд (UNICODE тэмдэгтүүдийн дурын дараалал) орно. Нийлмэл өгөгдлийн бүтэцэд нэг төрлийн бус төрөлтэй массивууд (формат нь массивын элементүүдийг ижил төрлийн байхыг шаарддаггүй) болон түлхүүрүүд нь тэмдэгт мөр (бичлэгийн төрлүүдтэй төстэй) байдаг түлхүүр-утга (key-value) хосын цуглуулга болох объектуудыг агуулдаг. Жишээ болгон бид JSON бүтэц дэх нийлмэл болон үндсэн өгөгдлийн төрлүүдийг хольж харуулахын тулд харьцангуй төвөгтэй өгөгдлийн бүтцийг доор мэдээлэв.

```
{
  "name": "Argo",
  "crew": [
    { "name": "Jason",      "role": "Captain" },
    { "name": "Heracles",  "role": [ "Mastman", "Ballast" ] },
    { "name": "Tiphys",    "role": "Helmsman" },
    { "name": "Atalanta", "role": [ "Pitman", "Ballast" ] },
    { "name": "Argos",     "role": [ "Shipwright", "Ballast" ] }
  ],
  "need repair": false,
  "cargo hold": 5,
  "cargo": [ "goat", "golden fleece", "dragon's teeth" ]
}
```

Дээр бид хамгийн гадна талын хос бүржгар хаалтанд элементүүд нь хаагдсан хөлөг онгоцыг дүрсэлсэн объектыг тодорхойлно. Элементүүдэд хөлөг онгоцны name ("Argo" гэсэн тэмдэгт мөр), багийн гишүүдийн нэр болон үүрэг бүхий объектуудын дарааллыг агуулсан crew массив (дөрвөлжин хаалтанд бичсэн) орно (хөлөг онгоцонд гишүүн нэгээс олон үүрэг гүйцэтгэх тохиолдолд мөр эсвэл мөрийн массив).

XML нь JSON-ээс илүү энгийн төрлүүдийг ашигладаг: XML өгөгдлийн аль ч хэсэг нь тэмдэгтүүдийн мөр юм. Гэсэн хэдий ч, формат нь хөгжүүлэгчдэд өгөгдөл доторх (эсвэл лавлагаа) өгөгдлийн төрлийг тодорхойлох замаар XML форматтай өгөгдлийг хязгаарлаж, баталгаажуулах боломжийг олгодог.

JSON-той адил XML нь "эцэг" загвартай бөгөөд энэ нь 1980-аад онд стандартчилагдсан баримт бичиг хуваалцах хэл болох Standard Generalized Markup Language (SGML) -ээс санаа авч, XML нь 1995 он хүртэл, JSON нь 2000-аад оны эхэн үеэс эхэлдэг.

JSON-тэй адил XML нь "эцэг" загвартай бөгөөд 1980-аад онд стандартчилагдсан баримт бичиг хуваалцах хэл болох Standard Generalized Markup Language (SGML) -ээс санаа авч, XML нь 1995 онд, JSON нь 2000-аад оны эхэн үеэс эхэлдэг.

JSON-ийн нэгэн адил XML нь өгөгдлийг мод хэлбэртэй баримт бичиг гэж тодорхойлдог бөгөөд таг-ууд нь өгөгдлийн бүтцийг төлөөлж, тэдгээрийн дотор тэмдэгт мөр кодлогдсон өгөгдлийг агуулдаг. Жишээлбэл, `< name > Argo </ name >` мөр нь Argo мөрийн утгыг агуулсан name (non-empty-element XML таг нь үргэлж start-tag `< TAG_NAME >` end-tag `</ TAG_NAME >` хэлбэртэй байдаг) нэртэй таг бүхий документийн хэсэг юм. Үүний дээд хэсэгт XML баримтууд нь өгөгдлийн төрлүүдийн форматыг тодорхойлсон Data Type Definitions (DTD) гэж нэрлэдэг толгой хэсгийг агуулж болох бөгөөд програмистууд үүнийг таг-д агуулагдах утгыг хязгаарлахад ашиглаж болдог - жишээ нь, таг-ын агуулгыг хөвөгч цэгийн тоо гэж тодорхойлох. Бүрэн байдал (completeness)-ыг хангахын тулд бид дээр үзүүлсэн JSON форматтай өгөгдлийн бүтцийн XML форматтай хөрвүүлэлтийг доор үзүүлж байна (ялангуяа ижил тагны дараалсан давталтаас массивууд хэрхэн үүсдэгийг анхаарна уу, жишээлбэл, `< crew >< /crew >` ба `< үүрэг >< / үүрэг >`).

```
< ship>
  < name>Argo</ name>
  < crew>
    < name>Jason</ name> < role>Captain</ role>
  </ crew>
  < crew>
    < name>Heracles</ name> < role>Mastman</ role> < role>Ballast</ role>
  </ crew>
  < crew>
    < name>Tiphys</ name> < role>Helmsman</ role>
  </ crew>
  < crew>
    < name>Atalanta</ name> < role>Pitman</ role> < role>Ballast</ role>
  </ crew>
  < crew>
    < name>Argos</ name> < role>Shipwright</ role> < role>Ballast</ role>
  </ crew>
  < need_repair>false</ need_repair>
  < cargo_hold>5</ cargo_hold>
  < cargo>goat</ cargo>
  < cargo>golden fleece</ cargo>
  < cargo>dragon's teeth</ cargo>
</ ship>
```

Схем ба өгөгдлийн төрлүүд

XML баримтууд нь DTD-г агуулж чаддаг нь үйлдлээс хүлээгдэж буй өгөгдлийн төрлийг (өгөгдлийн форматын технологид тусгайлан) зааж өгөхөд үйлчилгээ хандалгат программчлалын нөхцөлд онцгой ач холбогдолтой юм – хүчинтэй байдал болон баримт бичгийн аль алинд нь. Үнэн хэрэгтээ

XML нь DTD-ийн зэрэгцээ XML баримт бичгийн бүтэц, агуулгад хязгаарлалтыг илэрхийлж чаддаг өөр өөр схем хэлтэй байдаг - шинэ схемийн хэлүүд хөгжсөн гол шалтгаан нь DTD-ийн техникийн үзүүлэлтүүдийн дутагдлыг даван туулах явдал байв. Үүний нэгэн адил JSON нь хүлээгдэж буй (хүчинтэй) JSON утгуудын бүтцийг JSON-д суурилсан форматаар зааж өгөх боломжтой JSON схем зэрэг стандартуудыг олж авсан.

Жишээлбэл, доорх JSON схемийн хэсэг нь манай JSON бүтцэд зориулсан жишээг баталгаажуулах боломжтой.

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": { "type": "string" },
    "crew": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "name": { "type": "string" },
          "role": { "oneOf": [
            { "type": "string" },
            { "type": "array", "items": { "type": "string" } }
          ] }
        }
      }
    },
    "need repair": { "type": "boolean" },
    "cargo hold": { "type": "number" },
    "cargo": { "type": "array", "items": { "type": "string" } },
    "required": [ "name" ]
  }
}
```

Мөр тус бүрийн дэлгэрэнгүйг дурдалгүйгээр хэл шинжлэлийн үүднээс сонирхолтой хэсгүүдийг онцолж хэлье.

Эхлээд бид хүчинтэй эсэхийг шалгарчаа (validator) тодорхойлоход ашигладаг \$schema зааврын хувилбарыг зарладаг. Энэ нь манай программуудын төрлийг тодорхойлоход өгөгдсөн хэлний хэрэгжилтийн тусгай хувилбарыг ашиглахад ялгаатай биш юм (шинэ хувилбарууд нь шинэ боломжуудыг агуулж болно; жишээлбэл, Java 5.0-аас хойшхи генерик программуудыг оруулах, 10.4.2-ыг үзнэ үү). Схемийн технологи нь энэ хувилбарыг ил өгдөг бөгөөд ингэснээр схемийн аль ч хэрэглэгч үүнийг техникийн үзүүлэлтийн аль хувилбарын эсрэг ажиллуулах ёстойг мэддэг.

Дараа нь бид object болох бүтэн бүтцийн type-ийг зааж өгнө (JSON-ийн хувьд утгын харгалзуулалт хийх string-key). additionalProperties түлхүүр нь бидний тодорхойлж буй төрлийн дэд төрлүүдийг хүлээн авах эсэхийг заана; ялангуяа, энэ нь бүтцийн дэд төрлийн тайлбар юм (8.5.2-р хэсгийг харна уу), үүнд бид хүчинтэй утгуудыг схемд тодорхойлсоноос илүү олон талбар/шинж чанар байхыг зөвшөөрөх боломжтой. Энэ тохиолдолд бид схемд заасан бүтцийг өргөтгөхийг хориглоно (additionalProperties-ийг false гэж тохируулдаг).

Дараа нь бид хүчинтэй утгуудын properties-ыг олдог. Жишээлбэл, бид түлхүүрийн name нь string төрлийн утгатай тохирч байх ёстойг зааж өгдөг. Шинж чанаруудын тодорхойлолт нь ердийн

шаталсан төрлийн тодорхойлолтыг дагаж мөрддөг, жишээлбэл, бид crew шинэ чанар нь object-уудын array гэдгийг харуулж байгаа бөгөөд JSON схемийн объектуудад (properties хийгдсэн гэх мэт) заасан ижил конвенцийн дагуу үүнийг зааж өгдөг. crew массивын объектуудын тодорхойлолтод бид oneOf операторыг ашиглан role түлхүүрт тохирох утгуудын нийлбэрийн төрлийг (8.4.3-р хэсгийг үзнэ үү) тодорхойлдог бөгөөд энэ нь strings эсвэл strings-ийн arrays байж болохыг анхаарна үү.

Эцэст нь бид схемээ required түлхүүрээр хаадаг бөгөөд энэ нь JSON утгыг хүчинтэй гэж үзэхийн тулд аль шинж чанарууд байх ёстойг заадаг. Энэ тохиолдолд бид хүчинтэй утгууд нь хамгийн багадаа хамгийн гадна талын түлхүүрийн name-тэй байх ёстой гэж тохируулсан бөгөөд энэ нь бид хүчинтэй name-ийн утгатай, бүдүүвч дээрх бусад түлхүүрүүдийн тодорхойлолтуудын дурын хослолд нийцсэн аливаа JSON утгыг хүлээн авна гэсэн үг юм (бүтцийн дэд хэвшинжийн тухай боловсронгуй ойлголтыг хэрэглэх).

Жолигийн төрлүүд

Энэ хэсэг хүртэл бид өгөгдлийн төрлийг процессуудын хооронд өгөгдөл дамжуулахад ашигладаг технологитой нягт уялдуулан авч үзсэн. Жишээлбэл, бид JSON-г манай үйлчилгээнд ашигладаг сериалчлалын технологи болгон сонгож, солилцох өгөгдлийн төрлийг технологид тусгайлан зориулсан (technology-specific) хэлбэрээр тодорхойлж болно: JSON схем.

Гэсэн хэдий ч, бид харилцан ажиллахад чиглэсэн алхам хийсэн (мэдээлэл солилцох нийтлэг стандартыг бий болгосноор) бид сервисийнхээ үйлчлүүлэгчдийг JSON утгыг гаргах, задлахыг албадаж байгаа тул сул холболтын эсрэг амлалт өгсөн. Бусад үйлчлүүлэгчид зөвхөн хоёртын формат эсвэл XML-ийг сериалчлах технологи болгон ашиглах боломжтой бол яах вэ? Эдгээрийг манай үйлчилгээтэй харилцахыг зөвшөөрөх нь хоёр үйлдлийг шаарддаг; онолын ба практик. Онолын нь форматтай болгосон өгөгдлийг хүчинтэй эсэхийг шалгах схемийн тодорхойлолт (JSON хувилбаруудаас өөр, хэрэв байгаа бол) ба сэрвисээр үзүүлсэн интерфэйсийн өргөтгөлтэй холбоотой бөгөөд одоо өмнө нь үүссэн JSON суурьтай тодорхойлолтуудын хамт шинэ форматын бэлэн байдлыг нийтлэх ёстой. Мэдээжийн хэрэг практик арга хэмжээ бол манай үйлчилгээг өөрчлөх бөгөөд ингэснээр өгөгдлийг хүссэн форматыг ялгаж, санах ой руу хөрвүүлэх тусгай процедурыг хэрэгжүүлэх явдал юм.

Бидний харж байгаачлан одоо олон удаа тохиолддог (жишээ нь, бид бид нийтлэг, төрөл-агностик, өгөгдлийн бүтцийн тусгай хэрэгжилтийг тодорхойлох залхмаар болон алдаа гаргадаг (error-prone) даалгавараас сэргийлэхийн тулд параметр полиморфизмыг хэргэлэх үед), хийсвэрлэл нь программистуудыг дээрх дэг журмын ачааллаас чөлөөлж чадна - үйлчилгээ нь шинэ өгөгдлийн форматыг дэмжих бүрд үүнийг тэсвэрлэх ёстой.

Энэ тохиолдолд бид өгөгдлийн форматын технологийн хэрэгжилтийн дэлгэрэнгүйгээс хийсвэрлэсэн үйлчилгээнийхээ интерфэйсийг тодорхойлоход хэл хэрэглэх бөгөөд төрлийг тодорхойлох нийтлэг схемийг гаргаж болно. Энэхүү хийсвэрлэлийн үндсэн баталгаа бол тухайн хэлний ажиллах цагаар дэмжигддэг төрөл бүрийн технологид нийцсэн схемүүдийг гаргаж, хэрэглэгчдэд ямар шийдэл нь тэдний хэрэгцээнд хамгийн сайн тохирохыг шийдэх, эсвэл хүчинтэй эсэхийг шалгагчаар хийдэгтэй ижил шалгалтыг хэрэгжүүлэх замаар тэдгээрээс татгалзах боломжийг олгодог.

Альтернатив хувилбар хоёулаа давуу болон сул талуудтай.

Хоёр дахь нь өгөгдлийн формат өгөгдөөгүй үед ч схем хэлээр хангах боломжтой бөгөөд хэрэглэгчид ижил өгөгдлийн форматтай хийсвэрлэх технологид хандах боломжтой бол, тэд хийсвэр схемийг ашиглан аль ч дэмжигдсэн өгөгдлийн форматаар сериалчилсан өгөгдлийг гаргаж, хүчинтэй эсэхийг шалгах боломжтой. Үүний эсрэгээр, хоёр дахь сонголтыг ашиглах боломжгүй (жишээ нь, энэ нь өгөгдсөн технологийн стекийн дагуу биш бол) үед үүсгэсэн, форматад тохиромжтой схемүүд нь хийсвэрлэлээс үл хамааран хэрэгжүүлэгчид хүчинтэй эсэхийг шалгах боломжийг олгоно - Энэ нь зорилтот өгөгдлийн форматад тохирсон схем байгаа гэсэн үг бөгөөд үйлчилгээний төрөл өөрчлөгдөх бүрт технологид тохиромжтой схемийг дахин үүсгэж, дахин хуваарилах зардал гардаг.

Тодорхой жишээ болгон Жолигийн төрлийг авч үзье. Жолид дээр дурдсан хоёр дахь хувилбарыг хэрэгжүүлдэг, тухайлбал, энэ нь өгөгдөл дамжуулахад ашигладаг өгөгдлийн форматаас хийсвэр хэлбэрийн тодорхойлолттой үйлчилгээ хандлагат програмчлалын хэл (анх 2006 онд гарсан) юм.

Жолигийн төрлүүд нь бүтцийг өгөгдлийн модоор тодорхойлдог бөгөөд бидний JSON болон XML форматын хувьд харсантай маш ойрхон байдаг; Эдгээр модны үндэс нь зарим суурь төрлийн утгыг (boolean, integer, string гэх мэт) агуулж байдаг ба зангилаа бүр нь модны массивыг заадаг. Өмнөх жишээнүүдийн JSON болон XML-д суурилсан бүтцэд зориулсан Жолигийн өгөгдлийн төрөл нь дээр дурдсан JSON схемийн семантикийг мөн дагадаг бөгөөд дараах байдалтай байна.

```
type ship: {
  name: string
  crew*: {
    name?: string
    role*: string
  }
  need_repair?: bool
  cargo_hold?: int
  cargo*: string
}
```

Жолигийн програмчлалын хэл

Жоли бол програмчлалын хэл бөгөөд хөгжүүлэгчдэд энгийн, бүрдэл байдлаар үйлчилгээг программчлах боломжийг олгох зорилготой юм. Энэ бүлэгт бид Жоли программын үндсэн элементүүдийг (төрөл, интерфэйс, портч, үйлчилгээ, ажиллагааны орчин) шинжилж, энэ нь интерфэйс, порт болон хуваарилагдсан комплекс программын ажиллагааны орчинг тодорхойлох энгийн синтакстай болохыг харуулав. Эдгээр "суурь" шинж чанаруудаас гадна хэл нь хуваарилагдсан комплекс системийг бүтээхэд програмистуудад туслах дэвшилтэт бүтцийг санал болгодог.

Жишээлбэл, Жоли нь хураангуй синтакс ашиглан үйлчилгээний комплеьс бүрдлийг бий болгоход хэрэглэгчдэд тусалдаг архитектур программчлалын багцтай. Эдгээр бүтээлтийн зарим жишээ нь: "embedding" бөгөөд энэ нь өөр дотроо үйлчилгээг ажиллуулах боломжийг хөгжүүлэгчдэд олгодог бөгөөд үр ашигтай (жишээ нь, дотоод санах ойгоор харилцах) бүрдэл үйлчилгээг бий болгодог (портын тодорхойлолтын хэд хэдэн мөрийг өөрчилснөөр embedder үйлчилгээг embedder-ээс нь "салгаж" болдог учраас уг бүтэц нь ашигтай байдаг, 15.2.4-р хэсгийг үзнэ үү); "redirection" ба "aggregation" нь програмистуудад мессежийг үйлчилгээ хооронд бүтэцлэгдсэн байдлаар явуулах боломжийг олгодог – redirect болон aggregation хоёрын гол ялгаа нь эхнийх нь үйлчлүүлэгчдэд мэдэгдэж байгаа бөгөөд энэ нь аль дахин чиглүүлсэн үйлчилгээтэй харилцахыг хүсч байгаагаа

тусгайлан зааж өгөх шаардлагатай байдаг бол aggregation нь далд бөгөөд үйлчлүүлэгч нь aggregator-тэй харилцаж байгаа эсэхээ мэдэхгүй байх явдал юм; "courier" бөгөөд энэ нь мессежийг шалгах, өөрчлөх механизмаар aggregation-ийг хүлээн аваг дээр баяжуулдаг (хүсэлтийг зорилтот үйлчилгээ рүүгээ дамжуулахаас өмнө болон хариуг үйлчлүүлэгчид буцааж өгөхөөс өмнө).

Хэлний өөр нэг чухал шинж чанар (ялангуяа ажиллагааны орчны хувьд) нь доголдолтой үед системийг жигд ажиллуулах дэвшилтэт боломжуудыг агуулсан алдаа боловсруулалттай (15.2.5-д товч жишээ) байдаг. Ялангуяа Жоли алдааг хамрах хүрээний тусламжтайгаар зохицуулдаг. Хамрах хүрээ бол хамтад нь гүйцэтгэх ёстой кодын блокууд бөгөөд жишээлбэл, хамрах хүрээ нь гүйлгээний бүхлээрээ гүйцэтгэгдэх эсвэл эсрэгээрээ буцаагдэх хэсгийг тэмдэглэж болно. Энэ зорилгын үүднээс уг хэл нь "termination and compensation" механизмыг бий болгодог бөгөөд зогсох (termination) нь хөгжүүлэгчид үйлдэл амжилтгүй болсон бол юу хийхийг зааж өгөх боломжийг олгодог бол compensation нь амжилтгүй болсон үйлдлийн нөлөөг буцаах шаардлагатай үйлдлүүдийг зааж өгөх боломжийг олгодог.

Энэ хэл нь үйлдвэржилтэнд хүчтэй төвлөрдөг (жишээ нь, түүний синтакс нь Java, C хэлийг санагдуулдаг бөгөөд энэ нь өгөгдлийн формат болон харилцааны протоколын өсөн нэмэгдэх массивыг дэмждэг) ба хөдөлшгүй академик үндэслэлтэй - түүний анхны формал үндэс суурь, хэрэгжилт нь Болоньягийн их сургуулиас гаралтай бөгөөд одоогоор олон улсын судалгаа, үйлдвэрлэлийн байгууллагуудын оролцоотойгоор Өмнөд Данийн Их Сургууль төслийг удирдаж байна.

Тодорхойлолтод бид type ship-ээ нэрлэсэн. ship-ийн үндэс нь хоосон (void) бөгөөд үүнд name, crew, need_repair, cargo_hold, cargo зэргийг зангилаа болгон агуулна. ship-д name нь string төрөлтэй бөгөөд энэ нь бүтэн бүтцийн цорын ганц шаардагдах зангилаа юм. Энэ нь бусад зангилааны хажууд байгаа ? ба * операторуудаас тодорхой харагдаж байгаа бөгөөд зангилаа бүрт харгалзах массивын элементүүдийн тоо хэмжээг илтгэнэ - харгалзан **zero-or-one** эсвэл **zero-or-more** хэмжээтэй бол тоо хэмжээний тодорхойлогч байхгүй нь exactly-one шаардлагыг илэрхийлдэг (хамгийн гадна талын name гэх мэт).

Сонирхолтой нь, зангилаа нь массив учраас JSON схемийг ашигласан тодорхойлолтоос илүү энгийн тодорхойлолттой болсон. Үнэн хэрэгтээ, role*: string-ийн тусламжтайгаар бид crew-ийн role шинж чанар нь сонголттой (optional) бөгөөд хэрэв өгөгдвөл энэ нь тэмдэгт мөр (жишээ нь, унар массив) эсвэл массив тэмдэгт мөр агуулж болно, өөрөөр хэлбэл Жоли нийлбэрийн төрлүүдээс сонголтоо хийж болно.

Өөр нэг сонирхолтой зүйл бол JSON схем болон Жолигийн хооронд дэд төрөл дээрх дизайны сонголтыг тэмдэглэх явдал юм. Эхнийх нь уг төрлийн "нээлттэй" тайлбарыг ашигладаг бөгөөд үүнд additionalProperties байгаа эсэхийг тодорхой хориглож, ямар элементүүд шаардлагатайг зааж өгөх шаардлагатай. Сүүлийнх нь төрлүүдийн "хаалттай" хөрвүүлэлтийг дагаж мөрддөг бөгөөд анхдагч утга нь exactly-one тоо хэмжээг илэрхийлдэг ба дэд төрлийг тодорхой зааж өгсөн байдаг, жишээлбэл, нийлбэрийн төрлүүд болон undefined (энд харуулаагүй) гэж тэмдэглэгдсэн зангилаанууд. Уншигчийг дадлага болгон аль ч аргын давуу болон сул талуудын талаар тунгаан бодохыг урьж байна (жишээ нь, үйлчилгээ ба/эсвэл үйлчлүүлэгчийг хэрэгжүүлж буй хэрэглэгч байр сууринаас).

Энэ бүлгийн сүүлд (15.2.4-т) харуулснаар Жоли портууд нь тухайн хэлийг хэрэглэгчдэдээ data-format-abstract төрлүүдийг санал болгох боломжийг олгодог. Үнэндээ хэрэглэгчид өгөгдөл хүлээн авах/дамжуулах бодит форматыг (жишээ нь, JSON) портуудад тодорхойлдог бөгөөд Жилигийн ажиллах цаг (runtime) нь өгөгдлийн дотоод дүрслэлийн хувиргалтыг боловсруулдаг. Үүнд сонгосон өгөгдлийн форматад зориулсан схемийн хэл байгаа эсэхээс үл хамааран хүлээн авах/дамжуулах өгөгдлийн хүчинтэй эсэхийн шалгалт орно.

15.2.3 Мессеж илгээх паттерн

Үйлчилгээнүүд нь ямар нэгэн дундын орчноор харилцдаг процессууд учраас өгөгдлийн форматыг зөвшилцөхөөс гадна дуудалтыг бие биедээ хэрхэн дамжуулахаа тогтоох хэрэгтэй. Үйлчилгээ хоорондын дуудалт болон өгөгдөл дамжуулах үйлдлийг тодорхойлох ерөнхий арга нь гурван түвшнийг хамардаг.

Эдгээрийн хамгийн доод түвшин нь мэдээлэл дамжуулахад ашигладаг дундын орчинг (жишээлбэл, утастай эсвэл утасгүй холболт) тодорхойлдог *харилцааны сувгууд (communication channels)* юм. Жишээ нь, Ethernet болон Bluetooth стандартууд. Энэ стандартууд нь хэлтэй, ялангуяа хэлний түвшний хийсвэрлэлтэй ямар ч холбоогүй тул бид судалгаандаа нэмэлт тайлбар өгөхгүй.

Дараа нь бид өгөгдсөн харилцааны сувгаар хоёр ба түүнээс дээш оролцог талууд хэрхэн мэдээлэл солилцохыг тодорхойлсон *харилцааны протоколууд*-ыг хардаг. Харилцааны протокол нь харилцааны одоогийн төлөв байдлын хариуд харилцан солилцсон мессежийн синтакс, талуудын хоорондох дараалал, мессеж бүрийг хүлээн авахад хамаарах дүрмийг тогтоодог (дэглэлтийн above-all-parties хийсвэрлэлийг санагдуулах байдлаар харилцан үйлчлэлийг байнга илэрхийлдэг жишиг үзүүлэлтүүдтэй). Жишээлбэл, хэрэв харилцааны явцад талуудын аль нэг нь эвдэрсэн өгөгдлийг хүлээн авч, хэрэглэгдсэн харилцааны протокол нь алдааг сэргээхэд чиглэгдсэн бол протоколын хэрэгжилт нь гэмтсэн мессежийг олж илрүүлж, энэ тохиолдолд түүний зааврыг баримтална - жишээлбэл, илгээгчээс мессежийг дахин дамжуулахыг хүсэх. Харилцааны төлөөлөгч протоколууд нь Transport Communication Protocol (TCP) ба User Datagram Protocol (UDP) юм. Бид харилцааны сувгуудтай ижил шалтгаанаар эдгээрийн талаар тайлбар өгөхөөс зайлсхийдэг, тухайлбал, протоколууд нь манай хэлний талаархи танилцуулгад тайлбар хийх ач холбогдолтой шинж чанартай байдаггүй, гэхдээ тэдгээр нь гарч ирэх тул бид энд дурдлаа - энэ бүлгийн дараа - үйлчилгээний тодорхойлолтын элемент/параметруудийг тодорхойлох.

Сүүлийн түвшин бол талууд хооронд мессеж солилцох логиктой холбоотой мессеж илгээх паттерн (messaging pattern) юм. Эдгээр олон хэв маяг байгаа хэдий ч бид дараахь үндсэн загваруудад (мөн тэдгээрийн зарим хувилбарт) анхаарлаа хандуулав.

Хүсэлт-Хариултын паттерн (Request-Response Pattern)

Энэ бүлгийн оршил хэсэгт дурьдсанчлан, бид стек дээр суурилсан хяналт-шилжүүлэх загварыг танилцуулахдаа request-response (мөн request-reply гэж нэрлэдэг) загварыг аль хэдийн олж мэдсэн (Бүлэг 7-г үзнэ үү). Паттерн нь хамгийн түгээмэл загваруудын нэг (хэрэв биш бол) бөгөөд жишээлбэл, энэ нь вэбийн үндсэн ажиллагааны орчныг тодорхойлдог. Үнэн хэрэгтээ HTTP (hypermedia мэдээллийн систем дэх мессеж солилцох процессийг тодорхойлдог стандарт) нь вэб хөтчүүдийн вэб серверүүдтэй харилцах, нөөцийг (ерөнхийдөө вэб хуудас) удирдах арга замыг тодорхойлдог request-response-based протокол юм. HTTP нь мессеж дамжуулах request-response паттерн хэрэгжүүлдэг тул протокол дахь оролцогч талуудын хоорондын бүх харилцан үйлчлэл нь хүсэлт,

хариултын хослол хэлбэрээр явагддаг - ялангуяа вэб хөтөч нь зарим нөөц дээр хийгдсэн үйлдэлд зориулсан хүсэлтийг илгээдэг тал бөгөөд вэб сервер нь эдгээр хүсэлтийг боловсруулж, хариуг нь буцааж илгээдэг. Энэ загвар нь HTTP дээр алдаа хэрхэн ажилладагийг зааж өгдөг, өөрөөр хэлбэл бүх тохиолдолд сервер нь хөтчийн хүсэлтэд хариу өгөх ёстой бөгөөд үүнд алдаа гарсан тохиолдол ч багтана (серверээс буцаагддаг алдарт 4XX ба 5XX HTTP алдааны кодоор дамжуулан, хэрэв мессежийг хүргэхийг хориглох харилцааны түвшний зарим алдаа гарсанаас бусад тохиолдолд).

Хүсэлт-хариултын паттернийг ихэвчлэн синхрон загварын ангилалд хамааруулдаг, учир нь түүний логикийг ерөнхийд нь илгээгч ба хүлээн авагчийн хооронд синхрончлол гэж тайлбарладаг бөгөөд илгээгч болон хүлээн авагч хоёр бие биенийхээ мессежийг хүлээж байдаг (хүлээн авагч болон илгээгчид харгалзан хүсэлт, хариуг хүлээнэ). Гэсэн хэдий ч загвар нь хоёр мессежийн синхрон хэрэглээ гэсэн үг биш, харин хоёр талын харилцаа нь нэг чиглэлд болон хоёр чиглэлд хос мессежийг дамжуулахад хүргэдэг.

One-Way Pattern

Өөр нэг өргөн хүрээтэй мессежийн загвар бол илгээгч нь хүлээн авагчаас хариу хүлээхгүйгээр мессеж илгээдэг *нэг чиглэлт (one-way)* харилцаа юм. Мөн энэ тохиолдолд бид өөр өөр цаг хугацаанд *нэг чиглэлт* харилцааг тайлбарлаж болно. *Нэг чиглэлт* мессежийн ердийн тайлбар нь асинхрон байдаг тул илгээгч нь мессежээ дамжуулж, хүлээн авагч үүнийг хэрэглэсэн гэдгийг анхаардаггүй бөгөөд энэ нь харилцааны давхаргаас мессежийг зөв хүлээн авсан тухай мэдэгдлийг авах боломжийг үгүйсгэхгүй. Илүү ховор, синхрон тайлбараар бол хүлээн авагчаас мессеж ашиглалтын мэдээллийг хүлээдэг.

Нийтлэх-Захиалах загвар / Publish-Subscribe Pattern

Гурав дахь үндсэн мессежлэлтийн загвар нь *publish-subscribe* гэж нэрлэгддэг загвар юм. Энэ загварын өвөрмөц шинж чанар нь илгээгч болон хүлээн авагчид бие биенээ (байршлыг) мэддэггүй бөгөөд хоорондоо шууд мессеж илгээх, хүлээн авах боломжгүй байдаг. Үүний оронд *нийтлэгч (publisher)* гэж нэрлэгддэг мессеж илгээгч нь тухайн ангиллын нэг буюу хэд хэдэн ангитай холбодог. Мессеж хүлээн авагчид нь тухайн мессежтэй холбоотой аль нэг ангиудыг сонирхож байгаагаа илэрхийлсэн (өөрөөр хэлбэл бүртгүүлсэн) *захиалагч (subscribers)* гэж нэрлэгддэг нэгжүүд юм. Хэрэгжилтүүд нь харилцан адилгүй бөгөөд мессеж дамжуулахад зориулсан point-to-point (олон захиалагчдын зөвхөн нэг нь нийтлэгдсэн мессежийг хүлээн авдаг) семантикийг хэрэгжүүлэх боломжтой боловч паттерний хамгийн түгээмэл тайлбар нь олон дамжуулалтын (multicast) семантикийг дагадаг бөгөөд ангийн бүх захиалагчид тухайн ангитай холбоотой нийтлэгдсэн мессежийг хүлээн авах боломжтой байдаг. Ахисан түвшний хэрэгжүүлэлт нь хэрэглэгчдэд нийтлэх-захиалах семантикийн цаашдын өргөжүүлэлтийг тодорхойлох (жишээ нь, анги тус бүрээр) боломжийг олгодог, тухайлбал дамжуулалтын удирдамжтой. Энгийнээр хамгийн түгээмэл нь: *at-most once*, мессежийг захиалагчдад зөвхөн нэг удаа илгээдэг бөгөөд бодит хүлээн авагчийн баталгаажуулалтгүй; *at-least once*, захиалагчид ижил мессежийн нэг буюу хэд хэдэн хувийг хүлээн авах боломжтой бөгөөд дор хаяж нэг хувийг хүлээн авснаа баталгаажуулах хүртэл; *exactly once*, захиалагчийг байгаа эсэхийг шалгаж, мессежийг дамжуулж, түүнийг хэрэглэсэн тухай мэдэгдлийг хүлээн авах эсвэл хүлээн авах хүртэл өмнөх үйлдлүүдийг давтах баталгаажуулалтын протоколын ачаар захиалагч мессежийн зөвхөн нэг хувийг хүлээн авдаг.

Хэрэгжүүлэлтийн хувьд, нийтлэх-захиалах мессеж нь ихэвчлэн *message broker* эсвэл *bus* хэлбэрээр гуравдагч тал (нийтлэн нийтлэгчид болон захиалагчдын хажууд) байхыг шаарддаг. Энэ этгээдийн үүрэг бол захиалгыг боловсруулах болон нийтлэгдсэн мессежийн илгээлт, хүргэлтийг зохицуулах явдал бөгөөд илүү ахисан түвшний брокерууд мессежийг удирдах, чиглүүлэхэд комплекс логикийг ашиглаж болно. Жишээлбэл, зарим брокерууд (жишээ нь, Апачи Кафка) захиалагчдад өөрийн сонирхлын ангиллыг мессеж бүр дээр хэрэглэх логик томъёо ашшиглан тодорхойлох боломжийг олгодог (тиймээс захиалга нь нэрлэсэн ангилалд хамаарахгүй, харин тухайн захиалагчийн хувьд мессежийг "сонирхолтой" гэж ангилах зарим нөхцлийг тодорхойлох болно). Эхний үеийн үйлчилгээ хандалгат архитектур болох Enterprise Service Bus - ESB (15.3.1-ээс харна уу) нь өгөгдөл хөрвүүлэх, протокол хувиргах зэрэг функцуудтэй ахисан түвшний bus юм.

15.2.4 Үйлдлүүд ба интерфэйсүүд

Хэлний үүднээс бид үйлдэл нь ямар мессежийн паттернийг дагаж мөрдөхийг хүлээж буйг интерфэйсттэй байхыг үйлчилгээнд илэрхийлэх боломжийг олгох хэрэгтэй. Өгөгдлийн төрлүүдээс харахад үйлчилгээнүүд энэ мэдээллийг үйлчлүүлэгчдэдээ хэрхэн хүргэдэг нь харилцан адилгүй байдаг ч бид эдгээр аргыг ил болон далд гэсэн хоёр бүлэгт хувааж болно.

Мессежлэлтийн ил паттерн / Explicit Messaging Patterns

Үйлчилгээ нь тэдний үйлдэл бүрийг тодорхойлдог мессежийн загварыг ил зааж өгөхийн тулд бидэнд энэ зорилгоор зарим хэл хэрэгтэй.

Үйлчилгээ хандлагад зориулсан энэ ангиллын хэлний анхны төлөөлөгч нь Web Services Description Language (WSDL) юм; 2000 оноос эхтэй, өгөгдлийн төрлүүд (голчлон XML-төрөлт документ гэх мэт), интерфэйсүүд болон үйлчилгээгээр хангадаг холбоо барих цэгүүдийг тодорхойлох чадвартай хэл (бид 15.2.4-р хэсэгт дүн шинжилгээ хийнэ). WSDL нь XML-д суурилсан дэлгэрэнгүй формат тул бид жишээгээ мессежийн загвар, интерфэйсийн чиглэсэн агуулгатай хэсгүүдээр хязгаарлаж, төрлийн тодорхойлолтыг харуулахаас зайлсхийдэг (энэ нь 15.2.2-т заасан дүрмийг дагаж мөрддөг).

```
<interface name="PortInterface">
  <operation name="lookupShip" pattern="http://www.w3.org/ns/wsd1/in-out">
    <input messageLabel="In" element="tns:string"/>
    <output messageLabel="Out" element="tns:ship"/>
    <fault name="ShipNotDocked" element="tns:string"/>
  </operation>
  <operation name="dock" pattern="http://www.w3.org/ns/wsd1/in-only">
    <input messageLabel="In" element="tns:ship"/>
  </operation>
</interface>
```

Дээрх хэсгээс бид `lookupShip` болон `dock` гэсэн хоёр үйлдэлтэй `PortInterface`-ийн тодорхойлолтыг авна. `lookupShip` үйлдэл нь хүсэлт-хариултын загварыг (in-out, WSDL хэлбэрээр) дагадаг бөгөөд илгээгч нь request-ээ хөлөг онгоцны нэрийг (string хэлбэрээр) илгээдэг бөгөөд үйлчилгээ нь ship өгөгдлийн бүтцээр хариу өгдөг (жишээ нь, 15.2.2-т үзүүлсэн XML бүтэц). `dock` үйлдэл нь нэг чиглэлт (in-only) бөгөөд ship өгөгдлийн бүтцийг хүлээн авдаг — үйлдэл нь хөлөг онгоцыг боомт руу залгах үйлдлийг дуурайдаг (simulates). `lookupShip` үйлдэл нь `docked` хөлөг онгоцны мэдээллийг авахыг дуурайдаг тул бид хайлтын явцад гарч болзошгүй алдааны талаар, тухайлбал, үйлчлүүлэгчийн хүссэн нэртэй хөлөг онгоц боомтод хэзээ ч `docked` болоогүй үед гарч

болох талаар мэдэгдэх шаардлагатай. ShipNotDocked fault-г searchShip үйлдэлтэй холбосноор бид WSDL интерфэйс дээрх энэ боломжийг мэдэгддэг. Бүрэн байдлын хувьд, WSDL нь "in-out" ба "in-only"-ээс гадна WSDL нь хэрэглээгээр хүлээн зөвшөөрөгдсөн "robust in-only" гэж нэрлэгддэг нэг чиглэлтэй, мөн "in-optional-out" гэсэн заавал биш request-response-тэй байдаг. Стандарт нь дээр дурдсан паттерний бусад гүйцээлтийг (нийтдээ 8) тодорхойлдог бөгөөд эдгээрийг харилцан үйлчлэлийн нөгөө талын нэмэлт интерфэйсийг тодорхойлоход ашиглаж болно.

Нэмэлт (болон easier-to-read байх) жишээ болгон дээр дурдсан интерфэйсийн Жоли хэл дээр бичсэн өөр хувилбарыг авч үзье.

```
interface PortInterface {  
    requestResponse: lookupShip( string )( ship ) throws ShipNotDocked( string )  
    oneWay: dock( ship )  
}
```

Өмнөх хэсэг дээр бид Jolie-ийн interface түлхүүр үгийг ашиглан өөрийн PortInterface-ийг тодорхойлсон бөгөөд үүнд WSDL жишээн дээрх хоёр үйлдэл багтсан болно: ShipNotDocked fault гаргаж болох requestResponse байх lookupShip болон oneWay dock. Одоогийн байдлаар, requestResponses болон oneWays нь тухайн хэлээр дэмжигдсэн цорын ганц мессежийн загвар бөгөөд 15.2.4-р хэсэгт авч үзсэн "notification" болон "solicit-response" гэж нэрлэгдэх гүйцээлтийг авч үзэхэд нийтдээ дөрөв болно.

Мессежийн далд паттерн / Implicit Messaging Patterns

Интерфэйсийн (мөн үүн доторх үйлдлүүд) мессеж паттерныг тодорхойлох далд арга нь өгөгдлийн тусгай формат дахь үйлчилгээний өгөгдлийн төрлийг (үйлдлүүд) тодорхойлохтой төстэй арга юм: бид түүний дэмждэг үйлдлүүдийн мессеж загварыг тохируулах технологийг сонгож, үйлчилгээний интерфэйсийг тодорхойлоход ашигладаг.

Энэ төрлийн тодорхойлолтуудын түгээмэл жишээ бол Representational State Transfer (REST) архитектурын хэв маяг юм. REST-ийн анхны төсөл (мянган жилийн эхээр HTTP1.1 стандарт дээр ажиллаж байсан Рой Филдингийн бичсэн) нь тухайн үйлчилгээний интерфэйсийг дүрслэх арга зам болсон - Application Programming Interface (API) гэж бас нэрлэдэг - Uniform Resource Locations (URL) хэрэглэдэг, HTTP аргууд (GET, POST, PUT гэх мэт) болон HTTP дэмждэг өгөгдлийн формат (жишээ нь, URL encoding, JSON, XML) ашигладаг. Энэхүү төслийн зорилго нь хөгжүүлэгчдэд HTTP-ийн хэрэглээнд нийцсэн үйлчилгээг хөгжүүлэхдээ одоо байгаа интернетийн дэд бүтцийг (жишээ нь, idempotent үйлдлүүдийг кэшлэх, transparent прокси ашиглах) ашиглах боломжийг олгох явдал байв (жишээ нь, HTTP-ийн ижил хязгаарлалтыг дагаж мөрддөг hypermedia өгөгдөл дамжуулахад). Санал болгогчийн хүсэлээс үл хамааран REST нь үйлчилгээний интерфэйсийг тодорхойлоход зориулагдсан бөгөөд голчлон SOAP (Simple Object Access Protocol)-ийн хүндрэлийг илүү хялбараар солих зорилгоор ашигладаг; SOAP нь өгөгдлийн төрөл, интерфэйсийн тодорхойлолт, мессеж илгээлт, peers топологийг багтаасан XML болон HTTP дээр суурилсан (сонирхолтой тохиолдлоор, нэрээр нь) мессежийн боловсронгуй протокол (протокол нь улмаар зөвхөн интерфэйс ба хандалтын тодорхойлолтод зориулагдсан WSDL-ийн тодорхойлолтыг үндэслэсэн). Үйлчилгээг (интерфэйс) "RESTful" гэж тодорхойлсоноор (REST стилийн шаардлагыг дагаж мөрддөг тул ингэж нэрлэдэг) - хөгжүүлэгч нь технологийн шаардлагын багцыг (жишээ нь, HTTP ашиглах гэх мэт) далд хэлбэрээр хийдэг төдийгүй тэдгээр үйлдлүүд нь стандартын тодорхойлолтын дагуу HTTP методуудтай харгалзаж, бүгд request-response загварыг дагаж мөрддөг.

Формат-онцлог ба формат-хийсвэр өгөгдлийн төрлийн тодорхойлолттой адил мессеж загварын тодорхойлолтууд нь ил болон далд хэлбэрийн хоорондох хийсвэрлэх шинж чанарыг харуулдаг. Жишээлбэл, хийсвэр WSDL эсвэл Жили интерфэйсийг өгвөл бид HTTP стекийг харилцааны протокол болгон ашиглах, шаардлагатай бол үйлдлүүдийг HTTP метод, нөөцөд харгалзуулах замаар хийсвэр RESTful хувилбарыг санал болгодог үйлчилгээний хэрэгжилтийг тодорхойлж болно (бидний 15.2.4-т хийсэн шиг). RESTful интерфэйсийг бусад технологид дасан зохицдог болгох (жишээ нь, хүсэлтийг нэг форматаас нөгөө формат руу хөрвүүлдэг дундын үйлчилгээгээр дамжуулан, ESB-ийн протоколын зуучлалын функционалын нэг хэсэг болгон хийдэгтэй адил) боломжтой боловч энэ стил нь мессеж загварын мэдээллийг өгдөггүй бөгөөд хүсэлт-хариултаас өөр хэв маягийг илэрхийлэх стандарт арга байхгүй.

REST нь үйлчилгээ хоорондын үялдаа холбоог нэмэгдүүлж (бие биетэйгээ харилцахын тулд бүгд REST "ярих" шаардлагатай) байгаа хэдий ч түүний энгийн байдал нь илүү комплекс хувилбаруудтай харьцуулахад давуу талтай бөгөөд үүнийг бичих нь үйлчилгээний интерфэйсийг тодорхойлох гол загваруудын нэг юм - тиймээс, жишээ нь, бага чадалтай, нөөцийн хязгаарлалттай төхөөрөмжүүдэд өргөнөөр хүлээн зөвшөөрөгдсөн харилцааны протокол болох Constrained Application Protocol (CoAP) нь RESTful үйлчилгээнд шууд дэмжлэг үзүүлэх (жишээ нь, үялдуулах), нэгтгэх зорилготой HTTP загварыг дагаж мөрддөг.

Bindings and Ports

Үйлчилгээний интерфэйсийг зааж өгөхдөө бид хоёр үндсэн сонголттой гэсэн ойлголтыг уншигч одоо олж авсан байж магадгүй юм.

Эхний, илүү тодорхой зүйл бол харилцааг хэрэгжүүлэхэд ашигласан бодит технологийн үүднээс интерфэйсийг шууд тайлбарлах явдал юм (жишээ нь, JSON өгөгдлийн форматыг сонгох, JSON схемээр төрлүүдийг тодорхойлох, үйлдлүүдийг RESTful стилээр тайлбарлах гэх мэт).

Хоёрдахь хувилбар, илүү хийсвэр (мөн сул үчлдаатай) нь үйлчилгээний хэрэгжилтийг тодорхойлдог бодит технологиос (жишээ нь, Жилигийн төрлүүд болон интерфэйсүүдийг ашиглах) хамааралгүй интерфэйсийг тодорхойлох боломжийг олгох хэлний бүтцийг ашиглах явдал юм. Аливаа хийсвэрлэлийн нэгэн адил бид үүнийг хэрхэн бодит байдалд ойртуулах, өөрөөр хэлбэл интерфэйсүүдийг хэрэгжүүлэхийн тулд ямар технологи ашиглахыг хүсч байгаагаа тодорхойлох боломжийг олгодог бүтэцүүдээр нөхөх хэрэгтэй. Эдгээр бүтэцүүд нь (a) өгөгдлийн формат болон харилцааны технологийн бодит хослолд интерфэйсийг *холбох (binding)* эсвэл харгалзуулах, (б) хэрэглэгчид үйлчилгээнд хэрхэн хүрч болохыг бодитоор тогтоодог холбох тусгай *port*-ын дэлгэрэнгүй мэдээлэл, жишээлбэл, үйлчилгээний сүлжээний хаяг, bus/мессеж брокерт холбогдох мэдээлэл гэх мэт.

WSDL-д дизайнерууд портын төрлүүдийн нэмэлт бүтцийг нэвтрүүлэх замаар интерфэйс, холболт болон порт (хэлний сүүлийн үеийн хувилбаруудад *төгсгөлийн цэг (endpoint)* гэж нэрлэдэг) хооронд илүү нарийн ялгах боломж сонгосон бөгөөд энэ нь ижил порт нэр дээр интерфэйс багцыг цуглуулдаг. Гэсэн хэдий ч, WSDL үйлчилгээг тайлбарлахын тулд (i) портын холболтыг өгөгдлийн формат болон харилцааны технологийн хослолд тодорхойлоход хангалттай; (ii) үйлчилгээг хэрэглэгчдэд үзүүлэхийн тулд технологи үндэслэлтэй мэдээлэл бүхий холболтыг бүрдүүлэх портыг зааж өгөх. Тиймээс WSDL *үйлчилгээ* нь нэгэн төрлийн бүс төгсгөлийн цэгүүд дээр интерфэйсүүдийн

цуглуулгын хэсгийг ил гаргах портуудын цуглуулгаас үүсдэг (өөр өөр мэдээллийн формат, харилцааны технологи ашиглан).

Жишээлбэл, бид PortInterface-г (15.2.4-р бүлгийг харна үү) RESTful үйлчилгээ болгон харуулсан WSDL < service > ашиглан “Harbour” жишээгээ гүйцээж болно.

```
<binding name="HttpBinding" interface="tns:PortInterface"
  type="http://www.w3.org/ns/wsdl/http">
  <operation ref="tns:dock" http:method="POST">
    <input> <mime:content type="application/xml" </input>
  </operation>
  <operation ref="tns:lookupShip" http:method="GET">
    <input> <mime:content type="application/xml" </input>
    <output> <mime:content type="application/xml" </output>
  </operation>
</binding>

<service name="Harbour" interface="tns:PortInterface">
<port name="RestEndpoint"
  binding="tns:HttpBinding"
  address="http://www.iolcus.com/harbour"/>
</service>
```

Дээрх хэсэгт HTTP холболт нь интерфэйсийг хэрэгжүүлэхэд ашигладаг технологийг зааж өгөхөөс илүү зүйлийг хийдэг гэдгийг анхаарна үү. Үнэн хэрэгтээ, уг холболт нь үйлдлүүдийг HTTP нөөцтэй харгалзуулж, тусгай HTTP методоор ашиглах боломжтой бөгөөд энэ нь солилцсон өгөгдлийн форматыг (xml) заадаг. Тодруулбал, dock зарим шинэ мэдээллийг хүлээн авдаг тул бид үүнийг POST рүү харгалзуулдаг (энэ метод нь нөөцөд зарим өгөгдлийг илгээх болон хүлээн авагчийн төлөвийн өөрчлөлтийг хоёуланг нь заадаг), бид lookupShip-ийг GET рүү харгалзуулдаг (мэдээлэл авах хүсэлт).

WSDL-ийн хувьд хэд хэдэн өөр дизайны сонголттой ч хийсвэр интерфэйсүүдийн холболт ба портуудын парадигм нь Жолид ч хамаатай. Нэгдүгээрт (15.2.4-т дурдсанчлан) WSDL нь өөрийн интерфэйсүүдэд мессежийн загвар (жишээ нь, нэг чиглэлт) болон харилцааны чиглэлийг хоёуланг нь зарладаг (жишээ нь, “in-only” эсвэл “out-only”, үйлчилгээ нь нэг чиглэлт мессежийг хүлээн авах эсвэл илгээх эсэхээс хамаардаг). Жолигийн хувьд интерфэйсүүд нь зөвхөн мессежийн загварыг заадаг бол харин энэ нь холболтын чиглэлийг тодорхойлдог интерфэйсүүд холбогдсон портын хэлбэр юм. Жишээлбэл, бид Жолид дээрхтэй ижил төгсгөлийн цэгийг дараах байдлаар тодорхойлж болно

```
inputPort HTTPport {
  location: "socket://www.iolcus.com:80/harbour"
  protocol: http {
    osc.dock.method      = "post"
    osc.lookupShip.method = "get"
    format = "xml"
  }
  interfaces: PortInterface
}
```

Энд бид inputPort-ыг тодорхойлсноор уг үйлчилгээ нь PortInterface-г дотогшоо чиглэсэн үйлдлүүд болгож байгааг харуулж байна. Үнэн хэрэгтээ Жоли хоёр төрлийн порттой: inputPorts болон

outputPorts нь тус порттой холбоотой interfaces цуглуулгад (интерфэйсийн нэрсийн таслалаар тусгаарлагдсан жагсаалтыг зааж өгч болно) байх үйлдлүүд нь дотогшоо эсвэл гадагш чиглэсэн байдаг гэдгийг харуулж байна.

WSDL-ийн хоёр дахь ялгаа нь Жоли портууд нь location, protocol тодорхойлолтыг агуулсан холболтын тухай ойлголтыг нэгтгэдэг явдал юм. location нь портын ашигладаг дундын орчин болон дамжуулалтын protocol хоёуланг нь заана. Дээрх жишээн дээрх угтвар угтвар socket:// нь уламжлалт HTTP серверийн порт 80 дээрх www.iolcus.com/harbour хаягаар холболтыг хүлээн авдаг Internet Protocol (IP) дээрх TCP сокетын хэрэглээг заадаг - WSDL нь HTTP холболтуудад TCP/IP стек гэж үзэн энэ элементийг хэсэгчлэн орхигдуулдаг. Портын protocol нь мессеж солилцох харилцаа, мэдээллийн форматын технологийг тодорхойлдог. Жишээн дээр бид http-г харилцааны протокол болгон зааж, (WSDL-ийн адил) интерфэйсийн харгалзуулалт хийх HTTP метод болон солилцсон өгөгдлийн формат нь xml байна. Жолигийн портууд болон интерфэйсүүдэд тэг хэмтэй хувилбар бол үйлчилгээ нь үйлдлийг хэрхэн ил гаргахыг өөрчлөхөд хялбар болгодог. Жишээлбэл, бид байршлыг "bt12cap: //" болгож өөрчилж, үйлчилгээгээ bluetooth хэрэгслээр дамжуулан дамжуулах боломжтой. Үүний нэгэн адил бид үйлчилгээ нь хүсэлтийг хүлээн авч, хариу өгөх бусад арга замыг дэмжихийн тулд protocol параметрийг өөрчилж болно. Жишээлбэл, бид ерөнхий http протоколоос илүү тусгайлсан jsonrpc, xmlrpc (HTTP ашигладаг боловч дурын өгөгдлийн форматын дээр үйлдэл, өгөгдлийн стандартыг тодорхойлдог) эсвэл зурвасын өргөнийг хэмнэхийн тулд soderp хоёртын формат руу шилжиж болно. Эдгээр бүх тохиолдолд, ийм өөрчлөлтүүд нь харилцан адилгүй интерфэйсүүдэд (мөн бидний үзэхээр, ажиллагааны орчин) нөлөөлөхгүй.

15.2.5 Ажилагааны орчин (Behavior)

Оркестржуулалт

Энэ бүлгийн оршил хэсэгт үзүүлснээр найрал хөгжим нь үйлчилгээнд чиглэсэн програмчлалд зан төлөвийг тодорхойлоход хамгийн тогтсон, ашиглагддаг парадигм юм. Бид түүний сэтгэл татам байдлыг хоёр баримттай холбож болно. Нэгдүгээрт, энэ арга нь дараалсан програмчлалын тархсан системийн байгалийн өргөтгөл юм (өөрөөр хэлбэл, бидний танилцуулж, 14-р бүлэг хүртэл ярилцсан. Хоёрдугаарт, бид дараагийн хэсэгт харж байгаачлан, бүжиг дэглэлтийн талаар нэмэлт арга барил нь тодорхой тайлбарыг олж чадаагүй байгаа бөгөөд энэ нь идэвхтэй судалгааны сэдэв хэвээр байна.

15.2.6 й